■ World's Best C++ & DSA Master Progress Sheet

The Ultimate Pedagogical Roadmap: Formatted as a Full-Featured Progress & Revision Tracking Sheet

■ Master Progress Summary & Metrics

Tracker Metric	Detail	Status		
Learning Path Steps	13 Ordered Steps (Foundations *to* Linear *to* Trees *to* Graphs *to* DP *to* Advanced)	■ Optimal Order		
Core Concepts & Theory	90+ In-Depth Theoretical Subtopics	[ ] Completed		
Curated Must-Do Problems	120 Industry-Standard Interview Problems (LeetCode / Striver A2Z)	[ ] 0 / 120 Solved		
Difficulty Breakdown	■ 30 Easy \	■ 65 Medium \	■ 25 Hard	■■ Balanced
Offline Version	Printable PDF Tracker (DSA_Syllabus.pdf)	■ Ready		

■■ Optimal Pedagogical Learning Order

```
flowchart TD
    Step0[Step 0: C++ Fundamentals & Memory] --> Step1[Step 1: Time & Space Complexity]
    Step1 --> Step2[Step 2: Math & Bit Manipulation]
    Step2 --> Step3[Step 3: Recursion & Backtracking]
    Step3 --> Step4[Step 4: Arrays & Strings]
    Step4 --> Step5[Step 5: Sorting & Binary Search]
    Step5 --> Step6[Step 6: Linked Lists]
    Step6 --> Step7[Step 7: Stacks & Queues]
    Step7 --> Step8[Step 8: Binary Trees & BST]
    Step8 --> Step9[Step 9: Heaps & Priority Queues]
    Step9 --> Step10[Step 10: Graph Algorithms]
    Step10 --> Step11[Step 11: Dynamic Programming]
    Step11 --> Step12[Step 12: Advanced Structures & Strings]
```

■ Step 0: C++ Language Fundamentals & Memory Layout

Pedagogical Rationale: *Before analyzing algorithms, you must master variable scopes, pointers, reference semantics, dynamic memory allocation, and the Standard Template Library (STL) to write efficient, low-overhead code.*

■ Theory & Language Mechanics

- ❏ Syntax, primitive data types, range overflow boundaries (*2^{31}-1* vs *2^{63}-1*).
- ❏ Fast I/O mechanics (**`ios_base::sync_with_stdio(false); cin.tie(NULL);`**).
- ❏ Pass-by-Value vs Pass-by-Reference (**`&`**) vs Pass-by-Const-Reference (**`const string&`**).
- ❏ Pointers, dereferencing (**`*`**), address-of (**`&`**), pointer arithmetic, and null pointers (**`nullptr`**).
- ❏ STL Sequence Containers: **`std::vector`** (dynamic array, capacity vs size, **`reserve()`**), **`std::deque`**, **`std::list`**.
- ❏ STL Associative Containers: **`std::set`**, **`std::map`** (Red-Black Trees, *O(\log N)*) vs **`std::unordered_set`**, **`std::unordered_map`** (Hash tables, average *O(1)*).
- ❏ STL Adaptors & Algorithms: **`std::stack`**, **`std::queue`**, **`std::priority_queue`**, **`std::sort`**, **`std::lower_bound`**, **`std::upper_bound`**.
- ❏ Advanced C++: Smart pointers (**`unique_ptr`**, **`shared_ptr`**), move semantics (**`&&`**), templates, and custom lambda comparators.

■ Step 1: Time & Space Complexity Analysis

Pedagogical Rationale: *Enables you to rigorously evaluate algorithm efficiency, prevent Time Limit Exceeded (TLE) errors, and choose optimal data structures.*

■ Theory & Complexity Rules

❏ Asymptotic Notations: Big-O (*O*), Big-Omega (*\Omega*), Big-Theta (*\Theta*).

❏ Time Complexity derivation for single loops, nested loops, logarithmic loops (*O(\log N)*), and exponential recursion (*O(2^N)*, *O(N!)*).

❏ Space Complexity: Auxiliary memory vs Total memory, Call Stack Depth in recursion.

❏ Input Size Constraint Analysis (Mapping *N* to required algorithm complexity):

- *N \le 10 \implies O(N!)* or *O(2^N)* (Backtracking / Bitmask)
- *N \le 10^2 \implies O(N^3)* (Floyd-Warshall / Matrix DP)
- *N \le 10^3 \implies O(N^2)* (2D DP / Nested Loops)
- *N \le 10^5 \implies O(N \log N)* (Sorting / Binary Search / Segment Tree)
- *N \le 10^8 \implies O(N)* or *O(\log N)* (Linear Scan / Binary Search)

■ Step 2: Essential Math & Bit Manipulation

Pedagogical Rationale: *Bitwise operations operate directly at the hardware register level, providing O(1) time state tracking and efficient mathematical problem-solving.*

■ Theory & Bit Formulas

❏ Binary numbers, Two's Complement representation, signed vs unsigned integer overflow.

❏ Bitwise operators: **&** (AND), **|** (OR), **^** (XOR), **~** (NOT), **<<** (Left Shift), **>>** (Right Shift).

❏ Core Bit Tricks:

- Check *i*-th bit set: `(n & (1 << i)) != 0`
- Set *i*-th bit: `n | (1 << i)`
- Clear *i*-th bit: `n & ~(1 << i)`
- Toggle *i*-th bit: `n ^ (1 << i)`
- Unset lowest set bit: `n & (n - 1)` (Brian Kernighan's Algorithm)
- Isolate lowest set bit: `n & (-n)`
- Check power of 2: `(n > 0) && ((n & (n - 1)) == 0)`

❏ Euclidean Algorithm for GCD & LCM, Prime Sieve of Eratosthenes (*O(N \log \log N)*).

■ Progress Tracker: Math & Bit Manipulation

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Single Number**	LeetCode #136	■ Easy	XOR Property	[ ] Completed

[ ]	**Number of 1 Bits**	LeetCode #191	■ Easy	Kernighan's Bit Trick	[ ] Completed
[ ]	**Counting Bits**	LeetCode #338	■ Easy	Bitwise DP	[ ] Completed
[ ]	**Single Number II & III**	LeetCode #137 / #260	■ Medium	Bitmasking & Bucket XOR	[ ] Completed
[ ]	**Reverse Bits**	LeetCode #190	■ Easy	Bit Shift Manipulation	[ ] Completed
[ ]	**Bitwise AND of Numbers Range**	LeetCode #201	■ Medium	Common Binary Prefix	[ ] Completed
[ ]	**Pow(x, n)**	LeetCode #50	■ Medium	Binary Exponentiation	[ ] Completed
[ ]	**Count Primes**	LeetCode #204	■ Medium	Sieve of Eratosthenes	[ ] Completed

■ Step 3: Recursion & Backtracking Fundamentals

Pedagogical Rationale: *Recursion builds the foundational mindset for state-space tree search, which is mandatory before mastering Tree traversals, Graph DFS, and Dynamic Programming.*

■ Theory & State Space Exploration

❏ Recursion tree visualization, Base Case definition, Recursive Relation, Call Stack mechanics.

❏ Backtracking Framework: **Choose *to* Explore *to* Un-choose (State Reset)**.

❏ Decision Patterns: Include/Exclude pattern for Power Set, Placement constraint validation for Grid search.

❏ Pruning state space to avoid Time Limit Exceeded (TLE).

■ Progress Tracker: Recursion & Backtracking

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Subsets I & II**	LeetCode #78 / #90	■ Medium	Include/Exclude & Sorting	[ ] Completed
[ ]	**Permutations I & II**	LeetCode #46 / #47	■ Medium	Visited Array / Swapping	[ ] Completed

[ ]	**Combination Sum I & II**	LeetCode #39 / #40	■ Medium	Backtracking & Pruning	[ ] Completed
[ ]	**Letter Combinations of Phone Number**	LeetCode #17	■ Medium	Mapping Tree Search	[ ] Completed
[ ]	**Word Search**	LeetCode #79	■ Medium	Matrix DFS Backtracking	[ ] Completed
[ ]	**Palindrome Partitioning**	LeetCode #131	■ Medium	String Split Backtracking	[ ] Completed
[ ]	**N-Queens**	LeetCode #51	■ Hard	Board Constraint Checking	[ ] Completed
[ ]	**Sudoku Solver**	LeetCode #37	■ Hard	Grid Constraint Search	[ ] Completed

---

■ Step 4: Arrays & Strings (Pointers & Windowing)

Pedagogical Rationale: *Arrays are contiguous memory structures. Mastering Two Pointers, Sliding Window, and Prefix Sum techniques reduces brute-force O(N^2) array problems to optimal O(N) linear time.*

■ Theory & Algorithmic Patterns

- ❏ 1D & 2D Array memory mapping (Row-Major vs Column-Major).
- ❏ **Two Pointers:** Opposite direction (2-Sum, Palindromes) vs Same direction (Fast/Slow pointers, In-place removal).
- ❏ **Sliding Window:** Fixed Window size *K* vs Dynamic Variable Window (expanding `right`, contracting `left`).
- ❏ **Prefix Sum & Difference Array:** *O(1)* Range Sum Querying & Range Updating.
- ❏ **Kadane's Algorithm:** Local optimal sum vs Global optimal sum state transition.
- ❏ **Dutch National Flag Algorithm:** 3-way partitioning in *O(N)* time and *O(1)* space.

■ Progress Tracker: Arrays & Strings

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed

[ ]	**Two Sum**	LeetCode #1	■ Easy	Hash Map Complement	[ ] Completed
[ ]	**Best Time to Buy & Sell Stock**	LeetCode #121	■ Easy	Minimum Tracking	[ ] Completed
[ ]	**Maximum Subarray**	LeetCode #53	■ Medium	Kadane's Algorithm	[ ] Completed
[ ]	**Sort Colors**	LeetCode #75	■ Medium	Dutch National Flag	[ ] Completed
[ ]	**3Sum**	LeetCode #15	■ Medium	Sorting + Two Pointers	[ ] Completed
[ ]	**Container With Most Water**	LeetCode #11	■ Medium	Shrinking Two Pointers	[ ] Completed
[ ]	**Subarray Sum Equals K**	LeetCode #560	■ Medium	Prefix Sum + Hash Map	[ ] Completed
[ ]	**Find All Anagrams in a String**	LeetCode #438	■ Medium	Fixed Sliding Window	[ ] Completed
[ ]	**Spiral Matrix**	LeetCode #54	■ Medium	2D Boundary Traversal	[ ] Completed
[ ]	**Rotate Image (2D Matrix)**	LeetCode #48	■ Medium	Transpose + Reverse	[ ] Completed
[ ]	**Trapping Rain Water**	LeetCode #42	■ Hard	Two Pointers / Monotonic	[ ] Completed
[ ]	**Minimum Window Substring**	LeetCode #76	■ Hard	Variable Sliding Window	[ ] Completed

■ Step 5: Sorting Algorithms & Binary Search

Pedagogical Rationale: *Searching on sorted data or monotonic answer spaces reduces search time from linear O(N) to logarithmic O(\log N), forming a core pillar of optimization.*

■ Theory & Monotonic Predicates

- ❏ Comparison vs Non-Comparison Sorts (Merge Sort $O(N \log N)$ Stable, Quick Sort $O(N \log N)$ Average, QuickSelect $O(N)$ Kth element).
- ❏ Binary Search Fundamentals: `low`, `high`, `mid = low + (high - low)/2` to avoid integer overflow.
- ❏ **Lower Bound** (`first idx >= target`) and **Upper Bound** (`first idx > target`).
- ❏ Rotated Sorted Array Searching (identifying the sorted half at every step).
- ❏ **Binary Search on Answer Space:** Monotonic Predicate Function $f(x) \in \{\text{True, False}\}$, shrinking search range `[low, high]`.

■ Progress Tracker: Sorting & Binary Search

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Binary Search**	LeetCode #704	■ Easy	Standard Binary Search	[ ] Completed
[ ]	**Search in Rotated Sorted Array**	LeetCode #33	■ Medium	Modified Binary Search	[ ] Completed
[ ]	**Find First & Last Position in Array**	LeetCode #34	■ Medium	Lower & Upper Bound	[ ] Completed
[ ]	**Kth Largest Element in an Array**	LeetCode #215	■ Medium	QuickSelect Algorithm	[ ] Completed
[ ]	**Search a 2D Matrix I & II**	LeetCode #74 / #240	■ Medium	Staircase Search / 2D BS	[ ] Completed
[ ]	**Find Peak Element**	LeetCode #162	■ Medium	Gradient Binary Search	[ ] Completed
[ ]	**Koko Eating Bananas**	LeetCode #875	■ Medium	BS on Answer Space	[ ] Completed
[ ]	**Capacity To Ship Packages in D Days**	LeetCode #1011	■ Medium	BS on Answer Space	[ ] Completed
[ ]	**Book Allocation / Aggressive Cows**	Standard Pattern	■ Medium	BS on Monotonic Answer	[ ] Completed

[ ]	**Median of Two Sorted Arrays**	LeetCode #4	■ Hard	Partition Binary Search	[ ] Completed

■ Step 6: Linked Lists (Pointers & Cache Layouts)

Pedagogical Rationale: *Linked Lists teach precise node pointer manipulation without contiguous array layout. Mastering Floyd's Cycle Detection and LRU cache lays the groundwork for graph node structures.*

■ Theory & Pointer Mechanics

❏ Memory Allocation: Non-contiguous heap nodes vs contiguous array memory layout.

❏ Variations: Singly, Doubly, Circular Linked Lists.

❏ **Floyd's Cycle Detection (Fast & Slow Pointers):** Proof of meeting point $2k - k = n \cdot C$ and entry node location.

❏ In-place Linked List Reversal (Iterative 3-Pointer vs Recursive).

❏ **Dummy Node Pattern:** Eliminating boundary null-pointer checks during head insertions/deletions.

❏ LRU / LFU Cache Architecture: Combining Hash Map ($O(1)$ lookups) + Doubly Linked List ($O(1)$ node repositioning).

■ Progress Tracker: Linked Lists

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Reverse Linked List**	LeetCode #206	■ Easy	Iterative / Recursive	[ ] Completed
[ ]	**Middle of the Linked List**	LeetCode #876	■ Easy	Fast & Slow Pointers	[ ] Completed
[ ]	**Merge Two Sorted Lists**	LeetCode #21	■ Easy	Two Pointers Merge	[ ] Completed
[ ]	**Linked List Cycle I & II**	LeetCode #141 / #142	■ Medium	Floyd's Cycle Detection	[ ] Completed
[ ]	**Remove Nth Node From End**	LeetCode #19	■ Medium	Two Pointer Offset	[ ] Completed

[ ]	**Reorder List**	LeetCode #143	■ Medium	Middle + Reverse + Merge	[ ] Completed
[ ]	**Add Two Numbers**	LeetCode #2	■ Medium	Digit Addition LL	[ ] Completed
[ ]	**Merge k Sorted Lists**	LeetCode #23	■ Hard	Min-Heap / Divide & Conquer	[ ] Completed
[ ]	**LRU Cache**	LeetCode #146	■ Medium	Hash Map + Doubly LL	[ ] Completed
[ ]	**LFU Cache**	LeetCode #460	■ Hard	Freq Map + Doubly LL	[ ] Completed

---

■ Step 7: Stacks & Queues (Monotonic Data Structures)

Pedagogical Rationale: *LIFO (Stack) and FIFO (Queue) control execution flows. Monotonic Stacks solve range-query problems (e.g. Next Greater Element) in amortized linear O(N) time.*

■ Theory & Monotonic Invariants

❏ Stack LIFO mechanics (Function Call Stack) vs Queue FIFO mechanics (Task queues).

❏ **Monotonic Stack Pattern:** Maintaining increasing/decreasing stack elements to find Next Greater Element (NGE) or Next Smaller Element (NSE).

❏ **Monotonic Queue Pattern:** Sliding Window Maximum tracking using Deque (*O(N)* time).

❏ Expression Parsing: Infix to Postfix conversion (Shunting-yard Algorithm) and Postfix Evaluation.

❏ Min Stack Architecture: *O(1)* space math encoding (`2 * val - minVal`) vs auxiliary min stack.

■ Progress Tracker: Stacks & Queues

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Valid Parentheses**	LeetCode #20	■ Easy	Stack Matching	[ ] Completed
[ ]	**Implement Queue using Stacks**	LeetCode #232	■ Easy	Amortized *O(1)* Stacks	[ ] Completed

[ ]	**Min Stack**	LeetCode #155	■ Medium	Auxiliary Stack / Math	[ ] Completed
[ ]	**Evaluate Reverse Polish Notation**	LeetCode #150	■ Medium	Postfix Evaluation	[ ] Completed
[ ]	**Daily Temperatures**	LeetCode #739	■ Medium	Monotonic Decreasing Stack	[ ] Completed
[ ]	**Next Greater Element I & II**	LeetCode #496 / #503	■ Medium	Monotonic Stack / Circular	[ ] Completed
[ ]	**Online Stock Span**	LeetCode #901	■ Medium	Monotonic Stack	[ ] Completed
[ ]	**Sliding Window Maximum**	LeetCode #239	■ Hard	Monotonic Deque	[ ] Completed
[ ]	**Largest Rectangle in Histogram**	LeetCode #84	■ Hard	Monotonic Stack Range	[ ] Completed
[ ]	**Basic Calculator I & II**	LeetCode #224 / #227	■ Hard	Stack Expression Parsing	[ ] Completed

■ Step 8: Trees & Binary Search Trees (Hierarchical Structures)

Pedagogical Rationale: *Trees introduce non-linear hierarchical branching. Mastering Tree DFS/BFS algorithms is essential before tackling complex Graph traversals and Tree Dynamic Programming.*

■ Theory & Traversals

❏ Tree Terminology: Root, Parent, Child, Leaf, Depth, Height, Ancestor, Subtree.

❏ Binary Tree Variants: Full, Complete, Perfect, Balanced, Degenerate.

❏ **Tree Traversals:**

- Depth-First Search (DFS): Inorder (L-Node-R), Preorder (Node-L-R), Postorder (L-R-Node) via Recursion & Iterative Stacks.
- Breadth-First Search (BFS): Level Order Traversal via Queue.

❏ **Lowest Common Ancestor (LCA):** Top-down split vs Bottom-up search logic.

❏ Binary Search Tree Invariant: $\text{Left Subtree} < \text{Node} < \text{Right Subtree}$. Inorder traversal of BST produces sorted order.

❏ BST Operations: Search, Insert, Delete (Leaf, 1-Child, 2-Children via Inorder Successor).

■ Progress Tracker: Trees & BST

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Maximum Depth of Binary Tree**	LeetCode #104	■ Easy	Simple DFS / BFS	[ ] Completed
[ ]	**Invert / Flip Binary Tree**	LeetCode #226	■ Easy	Recursive Postorder	[ ] Completed
[ ]	**Diameter of Binary Tree**	LeetCode #543	■ Easy	Postorder Global Max	[ ] Completed
[ ]	**Binary Tree Level Order Traversal**	LeetCode #102	■ Medium	Queue BFS Level	[ ] Completed
[ ]	**Lowest Common Ancestor of BT**	LeetCode #236	■ Medium	Recursive DFS LCA	[ ] Completed
[ ]	**Validate Binary Search Tree**	LeetCode #98	■ Medium	Range Checking / Inorder	[ ] Completed
[ ]	**Kth Smallest Element in BST**	LeetCode #230	■ Medium	Inorder Traversal	[ ] Completed
[ ]	**Construct Tree from Pre & Inorder**	LeetCode #105	■ Medium	Hash Map + Recursion	[ ] Completed
[ ]	**Binary Tree Maximum Path Sum**	LeetCode #124	■ Hard	Tree Dynamic Programming	[ ] Completed
[ ]	**Serialize & Deserialize BT**	LeetCode #297	■ Hard	Preorder / Level String	[ ] Completed

■■ Step 9: Priority Queues & Heaps (Order Statistics)

Pedagogical Rationale: *Binary Heaps maintain dynamic partial order in O(\log N) insertion/deletion and O(1) top retrieval, providing optimal solutions for Top-K and stream processing problems.*

■ Theory & Heapification

❏ Complete Binary Tree array representation (Parent `i/2`, Left `2i`, Right `2i+1`).

❏ Max-Heap vs Min-Heap properties.

❏ Building Heap (`Heapify`): *O(N)* time complexity mathematical proof vs *O(N \log N)* successive insertion.

❏ **Top-K Elements Pattern:** Maintaining Min-Heap of size *K* for *K* largest elements (*O(N \log K)*).

❏ **Two-Heaps Pattern:** Min-Heap + Max-Heap combination to track dynamic median in *O(1)* time.

■ Progress Tracker: Heaps & Priority Queues

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Kth Largest Element in a Stream**	LeetCode #703	■ Easy	Size-K Min-Heap	[ ] Completed
[ ]	**Top K Frequent Elements**	LeetCode #347	■ Medium	Min-Heap / Bucket Sort	[ ] Completed
[ ]	**K Closest Points to Origin**	LeetCode #973	■ Medium	Max-Heap / QuickSelect	[ ] Completed
[ ]	**Task Scheduler**	LeetCode #621	■ Medium	Max-Heap + Idle Queue	[ ] Completed
[ ]	**Reorganize String**	LeetCode #767	■ Medium	Greedy Max-Heap	[ ] Completed
[ ]	**Find Median from Data Stream**	LeetCode #295	■ Hard	Min-Heap + Max-Heap	[ ] Completed
[ ]	**Merge k Sorted Lists**	LeetCode #23	■ Hard	K-Way Min-Heap Merge	[ ] Completed

[ ]	**Smallest Range Covering K Lists**	LeetCode #632	■ Hard	Min-Heap + Sliding Window	[ ] Completed

■ Step 10: Graph Algorithms (Networks & Shortest Paths)

Pedagogical Rationale: *Graphs represent complex relationships (networks, routing, dependencies). Mastery of BFS, DFS, DSU, Dijkstra, and Topological Sort is mandatory for advanced systems engineering.*

■ Theory & Graph Algorithms

❏ Graph Representations: Adjacency Matrix (*O(V^2)* space) vs Adjacency List (*O(V + E)* space).

❏ Graph Traversals: BFS (Queue, shortest path in unweighted graphs) and DFS (Call stack, component discovery).

❏ Cycle Detection: Undirected (BFS/DFS parent check or DSU) vs Directed (DFS recursion stack or Kahn's in-degree check).

❏ **Topological Sort (DAG):** Kahn's Algorithm (BFS with In-degree array) and DFS Stack method.

❏ **Disjoint Set Union (DSU):** Path Compression & Union by Rank/Size (*O(\alpha(N))* amortized per operation).

❏ **Shortest Path Algorithms:**

- Dijkstra's Algorithm (Non-negative weights via Min-Heap, *O((V+E)\log V)*).
- Bellman-Ford Algorithm (Negative weights & Negative Cycle detection, *O(V \cdot E)*).
- Floyd-Warshall Algorithm (All-pairs shortest paths 2D DP, *O(V^3)*).

❏ **Minimum Spanning Trees (MST):** Kruskal's Algorithm (Greedy edges + DSU) & Prim's Algorithm (Greedy vertices + Min-Heap).

❏ Tarjan's Algorithm for Bridges & Articulation Points (Low-link values).

■ Progress Tracker: Graph Algorithms

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Number of Islands**	LeetCode #200	■ Medium	Grid BFS/DFS	[ ] Completed
[ ]	**Clone Graph**	LeetCode #133	■ Medium	Graph Map + BFS/DFS	[ ] Completed

[ ]	**Course Schedule I & II**	LeetCode #207 / #210	■ Medium	Topological Sort / Kahn's	[ ] Completed
[ ]	**Is Graph Bipartite?**	LeetCode #785	■ Medium	Graph Coloring BFS/DFS	[ ] Completed
[ ]	**Redundant Connection**	LeetCode #684	■ Medium	DSU Cycle Detection	[ ] Completed
[ ]	**Word Ladder**	LeetCode #127	■ Hard	Unweighted BFS Shortest	[ ] Completed
[ ]	**Network Delay Time**	LeetCode #743	■ Medium	Dijkstra's Algorithm	[ ] Completed
[ ]	**Cheapest Flights Within K Stops**	LeetCode #787	■ Medium	Bellman-Ford / Modified BS	[ ] Completed
[ ]	**Min Cost to Connect All Points**	LeetCode #1584	■ Medium	Kruskal's / Prim's MST	[ ] Completed
[ ]	**Critical Connections (Bridges)**	LeetCode #1192	■ Hard	Tarjan's Low-Link Algorithm	[ ] Completed

■ Step 11: Dynamic Programming (Optimal Substructure & Memoization)

Pedagogical Rationale: *DP optimizes exponential brute-force recursive calls (O(2^N)) into polynomial time (O(N) or O(N^2)) by caching overlapping subproblem answers.*

■ Theory & DP Paradigms

❏ Core Invariants: **Overlapping Subproblems** & **Optimal Substructure**.

❏ Approaches: Top-Down (Memoization with recursion + hash/array cache) vs Bottom-Up (Tabulation with iterative table).

❏ **Space Optimization:** Reducing state space from *O(N^2) \to O(N)* or *O(N) \to O(1)* by identifying minimum prior state dependencies.

❏ Major DP Frameworks:

- **1D DP:** Linear state transitions (Fibonacci, Staircase, House Robber).
- **Grid DP:** 2D cell paths (Unique Paths, Min Path Sum).

- **Knapsack DP:** 0/1 Knapsack (Single use) vs Unbounded Knapsack (Infinite copy).
- **String DP:** Longest Common Subsequence (LCS), Edit Distance.
- **LIS Pattern:** Longest Increasing Subsequence (*O(N^2)* DP vs *O(N \log N)* Binary Search Patience Sorting).
- **Interval DP / MCM:** Subsegment split optimization (Matrix Chain Multiplication, Burst Balloons).
- **Tree DP & Bitmask DP:** Subtree state passing & subset bitmask compression.

■ Progress Tracker: Dynamic Programming

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Climbing Stairs**	LeetCode #70	■ Easy	1D Fibonacci DP	[ ] Completed
[ ]	**House Robber I & II**	LeetCode #198 / #213	■ Medium	1D DP / Circular Array	[ ] Completed
[ ]	**Coin Change I & II**	LeetCode #322 / #518	■ Medium	Unbounded Knapsack DP	[ ] Completed
[ ]	**Partition Equal Subset Sum**	LeetCode #416	■ Medium	0/1 Knapsack Subset DP	[ ] Completed
[ ]	**Longest Common Subsequence**	LeetCode #1143	■ Medium	2D String Matching DP	[ ] Completed
[ ]	**Edit Distance**	LeetCode #72	■ Medium	2D String Edit DP	[ ] Completed
[ ]	**Longest Increasing Subsequence**	LeetCode #300	■ Medium	LIS *O(N \log N)* BS	[ ] Completed
[ ]	**Word Break**	LeetCode #139	■ Medium	1D String Partition DP	[ ] Completed
[ ]	**Unique Paths I & II**	LeetCode #62 / #63	■ Medium	2D Grid Traversal DP	[ ] Completed
[ ]	**Maximum Product Subarray**	LeetCode #152	■ Medium	1D Min/Max Tracking DP	[ ] Completed
[ ]	**Palindrome Partitioning II**	LeetCode #132	■ Hard	Min-Cut String DP	[ ] Completed

[ ]	**Burst Balloons**	LeetCode #312	■ Hard	Interval / MCM DP	[ ] Completed

■ Step 12: Advanced Data Structures & String Search

Pedagogical Rationale: *Advanced structures handle large-scale range queries, dynamic updates, and prefix matching in sub-linear time, required for high-frequency systems and competitive coding.*

■ Theory & Advanced Structures

❏ **Trie (Prefix Tree):** Multi-way tree structure for $O(L)$ word insertion, search, and prefix matching.

❏ **Segment Tree:** Complete binary tree for $O(\log N)$ range queries (Sum, Min, Max) and point/range updates.

❏ **Fenwick Tree (Binary Indexed Tree / BIT):** Prefix sum range queries and point updates in $O(\log N)$ time using bitwise low-bit manipulation (`idx & (-idx)`).

❏ **KMP (Knuth-Morris-Pratt) Algorithm:** Pattern matching in $O(N + M)$ time using Longest Prefix Suffix (LPS) array.

■ Progress Tracker: Advanced Structures & Strings

Done	Problem Title	Platform Link	Difficulty	Key Pattern	Status / Completed
[ ]	**Implement Trie (Prefix Tree)**	LeetCode #208	■ Medium	Trie Node Design	[ ] Completed
[ ]	**Design Add & Search Words**	LeetCode #211	■ Medium	Trie + Backtracking	[ ] Completed
[ ]	**Maximum XOR of Two Numbers**	LeetCode #421	■ Medium	Bitwise Trie	[ ] Completed
[ ]	**Range Sum Query - Mutable**	LeetCode #307	■ Medium	Segment Tree / BIT	[ ] Completed
[ ]	**Index of First Occurrence (KMP)**	LeetCode #28	■ Medium	KMP LPS Array	[ ] Completed
[ ]	**Shortest Palindrome**	LeetCode #214	■ Hard	KMP LPS Pattern Matching	[ ] Completed

■ Recommended 16-Week Progress Schedule

- ❏ **Weeks 1–2:** Step 0 (C++ Fundamentals & STL) & Step 1 (Complexity Analysis)
- ❏ **Weeks 3–4:** Step 2 (Math & Bit Operations) & Step 3 (Recursion & Backtracking)
- ❏ **Weeks 5–6:** Step 4 (Arrays, Two Pointers & Sliding Window)
- ❏ **Weeks 7–8:** Step 5 (Sorting & Binary Search) & Step 6 (Linked Lists)
- ❏ **Weeks 9–10:** Step 7 (Stacks & Queues) & Step 8 (Trees & Binary Search Trees)
- ❏ **Weeks 11–12:** Step 9 (Heaps & Priority Queues) & Step 10 (Graph Fundamentals)
- ❏ **Weeks 13–14:** Step 10 (Advanced Graph Shortest Paths/DSU) & Step 11 (Dynamic Programming)
- ❏ **Weeks 15–16:** Step 11 (Advanced DP Patterns) & Step 12 (Advanced Structures & Revision)